

Floating Point

COMP211 Connor McMahon

Microfloat

- **Microfloat** is a simplified, made-up floating-point representation that uses only **four bits**.

- The small number of bits allows us to easily explore design tradeoffs.

Key Idea: Design Tradeoffs

- There's a tradeoff between
  - The **range**: how large or small a number you can represent and
  - The **precision**: how accurately you can represent numbers
  - Both of which are constrained by the total number of available bits.

Key Idea: Design Tradeoffs

- More exponent bits
  - increases the range of representable numbers
  - but reduces the number of bits available for the significand, decreasing precision.
- More significand bits
  - increases the precision,
  - but reduce the number of bits available for the exponent, decreasing the range.

Microfloat Comparison

V1: 2 exp, 2 sig

Exponent	Significand	Value	Decimal	
00_2	00_2	$1.00_2 \times 2^0$	1	0.25
00_2	01_2	$1.01_2 \times 2^0$	1.25	
00_2	10_2	$1.10_2 \times 2^0$	1.5	
00_2	11_2	$1.11_2 \times 2^0$	1.75	
01_2	00_2	$1.00_2 \times 2^1$	2	0.5
01_2	01_2	$1.01_2 \times 2^1$	2.5	
01_2	10_2	$1.10_2 \times 2^1$	3	
01_2	11_2	$1.11_2 \times 2^1$	3.5	
10_2	00_2	$1.00_2 \times 2^2$	4	1
10_2	01_2	$1.01_2 \times 2^2$	5	
10_2	10_2	$1.10_2 \times 2^2$	6	
10_2	11_2	$1.11_2 \times 2^2$	7	
11_2	00_2	$1.00_2 \times 2^3$	8	2
11_2	01_2	$1.01_2 \times 2^3$	10	
11_2	10_2	$1.10_2 \times 2^3$	12	
11_2	11_2	$1.11_2 \times 2^3$	14	

V2: 1 exp, 3 sig

Exponent	Significand	Value	Decimal	
0_2	000_2	$1.000_2 \times 2^0$	1	0.125
0_2	001_2	$1.001_2 \times 2^0$	1.125	
0_2	010_2	$1.010_2 \times 2^0$	1.25	
0_2	011_2	$1.011_2 \times 2^0$	1.375	
0_2	100_2	$1.100_2 \times 2^0$	1.5	
0_2	101_2	$1.101_2 \times 2^0$	1.625	
0_2	110_2	$1.110_2 \times 2^0$	1.75	
0_2	111_2	$1.111_2 \times 2^0$	1.875	
1_2	000_2	$1.000_2 \times 2^1$	2	0.25
1_2	001_2	$1.001_2 \times 2^1$	2.25	
1_2	010_2	$1.010_2 \times 2^1$	2.5	
1_2	011_2	$1.011_2 \times 2^1$	2.75	
1_2	100_2	$1.100_2 \times 2^1$	3.5	
1_2	101_2	$1.101_2 \times 2^1$	3.25	
1_2	110_2	$1.110_2 \times 2^1$	3.5	
1_2	111_2	$1.111_2 \times 2^1$	3.75	

V3: 3 exp, 1 sig

Exponent	Significand	Value	Decimal	
000_2	0_2	$1.0_2 \times 2^0$	1	0.5
000_2	1_2	$1.1_2 \times 2^0$	1.5	
001_2	0_2	$1.0_2 \times 2^1$	2	1
001_2	1_2	$1.1_2 \times 2^1$	3	
010_2	0_2	$1.0_2 \times 2^2$	4	2
010_2	1_2	$1.1_2 \times 2^2$	6	
011_2	0_2	$1.0_2 \times 2^3$	8	4
011_2	1_2	$1.1_2 \times 2^3$	12	
100_2	0_2	$1.0_2 \times 2^4$	16	8
100_2	1_2	$1.1_2 \times 2^4$	24	
101_2	0_2	$1.0_2 \times 2^5$	32	16
101_2	1_2	$1.1_2 \times 2^5$	48	
110_2	0_2	$1.0_2 \times 2^6$	64	32
110_2	1_2	$1.1_2 \times 2^6$	96	
111_2	0_2	$1.0_2 \times 2^7$	128	64
111_2	1_2	$1.1_2 \times 2^7$	192	

Bias

- With microfloat V1, V2, and V3
  - We cannot represent values between 0 and 1
  - This is because the exponent must be **positive**

- **Solution:** Subtract a **fixed bias** from the exponent
  - Negative exponents allow us to represent values between 0 and 1

- For microfloat, the bias is 2
  - This allows for a range of [-2, 1]

Microfloat V4: Encoding 0.75

Decimal Place-Value

0	.	7	5
10^0		10^{-1}	10^{-2}

Binary Place-Value

0	.	1	1
2^0		2^{-1}	2^{-2}

Normalized Form

1.1×2^{-1}

Microfloat

0	1	1	0

Exponent Bits | Significand Bits

Exponent bits
-1 + bias = -1 + 2 = 1

Microfloat V4

Exponent	Significand	Value	Decimal
00_2	00_2	$1.00_2 \times 2^{0-2}$	0.25
00_2	01_2	$1.01_2 \times 2^{0-2}$	0.3125
00_2	10_2	$1.10_2 \times 2^{0-2}$	0.375
00_2	11_2	$1.11_2 \times 2^{0-2}$	0.4375
01_2	00_2	$1.00_2 \times 2^{1-2}$	0.5
01_2	01_2	$1.01_2 \times 2^{1-2}$	0.625
01_2	10_2	$1.10_2 \times 2^{1-2}$	0.75
01_2	11_2	$1.11_2 \times 2^{1-2}$	0.875
10_2	00_2	$1.00_2 \times 2^{2-2}$	1
10_2	01_2	$1.01_2 \times 2^{2-2}$	1.25
10_2	10_2	$1.10_2 \times 2^{2-2}$	1.5
10_2	11_2	$1.11_2 \times 2^{2-2}$	1.75
11_2	00_2	$1.00_2 \times 2^{3-2}$	2
11_2	01_2	$1.01_2 \times 2^{3-2}$	2.5
11_2	10_2	$1.10_2 \times 2^{3-2}$	3
11_2	11_2	$1.11_2 \times 2^{3-2}$	3.5

Distance between consecutive values

0.0625

0.125

0.25

0.5

Generalizing the Bias

$$\text{Bias} = 2^{n-1}-1$$

- n = number of bits in the exponent field

- This allows for almost the same number of positive and negative exponents
  - There is one more negative exponent

Bias

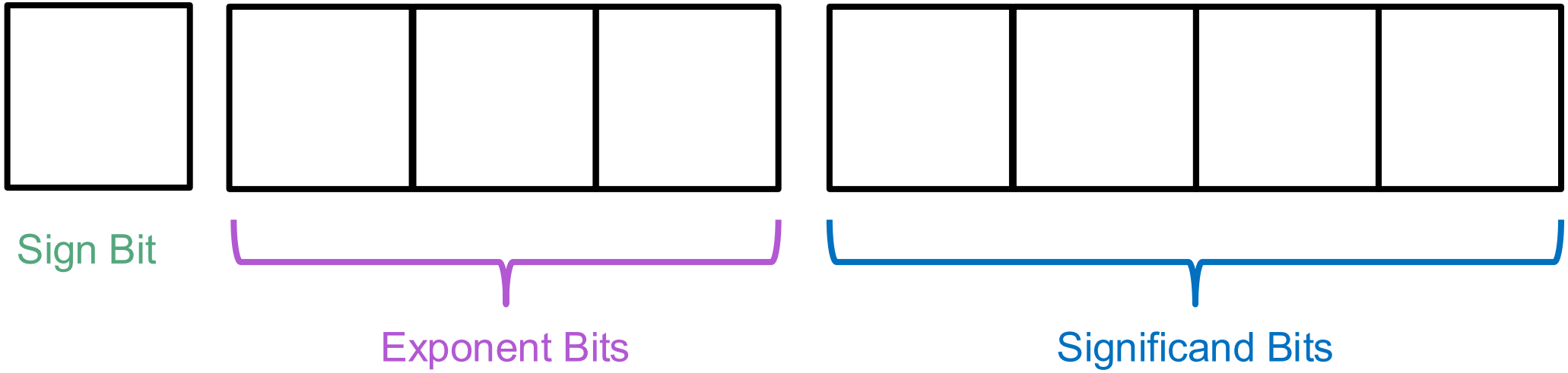
- Why do we use a bias instead of 2's complement?

- Simplifies Comparison Operations
  - With a biased exponent, larger numbers have higher bit values, and smaller numbers have lower bit values, making it easy to compare them as unsigned integers.
  - If two's complement were used, negative exponents (which have leading 1s) would be greater in unsigned comparison, breaking the ordering.

$$\text{Bias} = 2^{n-1}-1$$

Minifloat

Minifloat



Sign Bit (1 bit):
- 0: Positive number
- 1: Negative number

Exponent (3 bits):
- Represents the exponent using a fixed **bias of 3**

Significand (4 bits):
- Represents the fractional part
- Assumes an **implicit leading 1** for normalized numbers

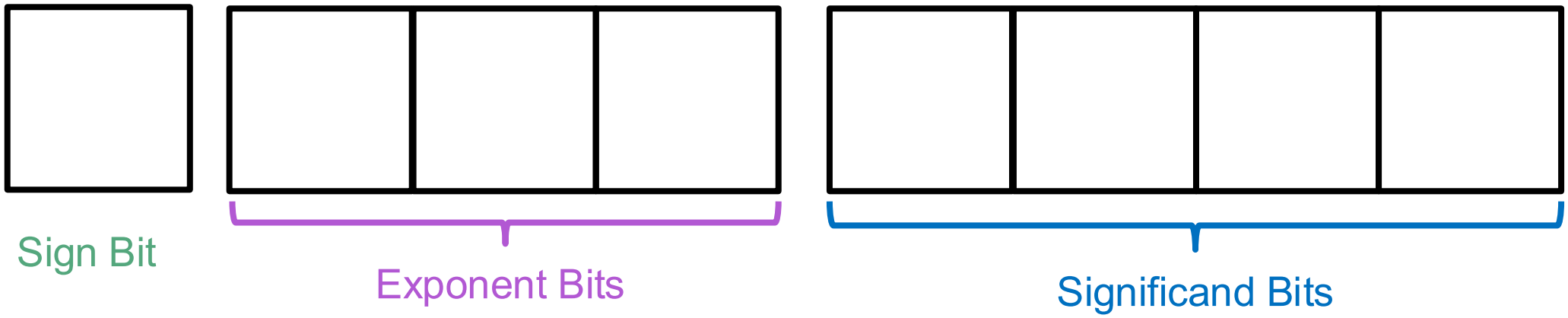
Decimal Place-Value

15

Binary Place-Value

Normalized Form

Minifloat



Decimal Place-Value

15

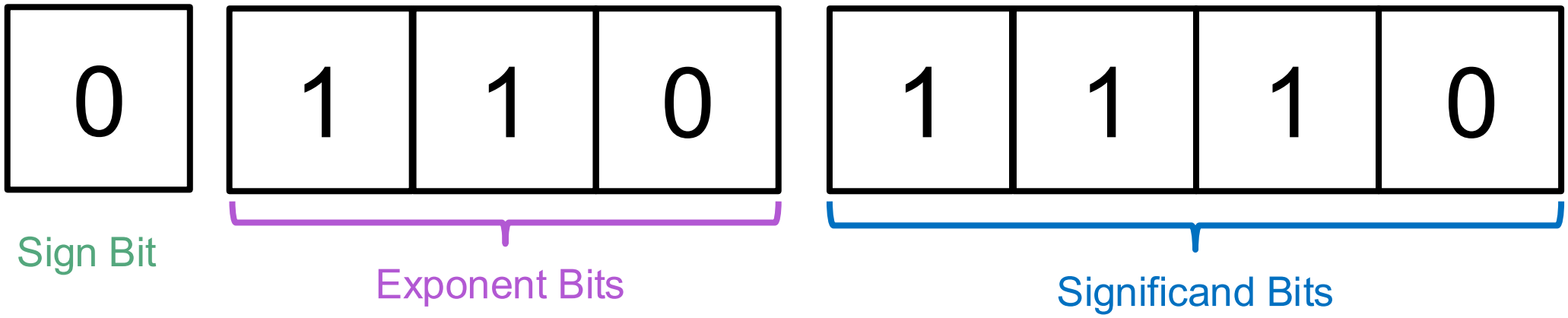
Binary Place-Value

1111_2

Normalized Form

1.111×2^3

Minifloat



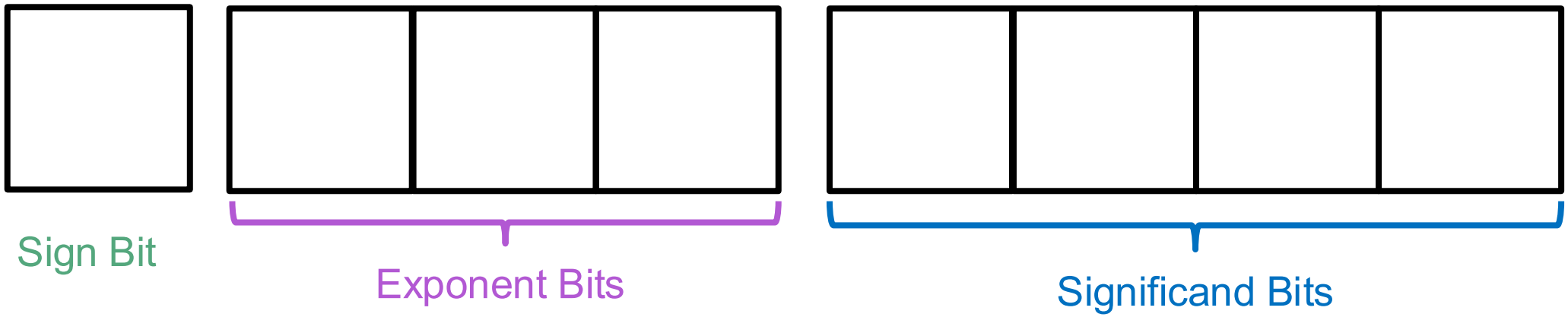
Decimal Place-Value

-4.25

Binary Place-Value

Normalized Form

Minifloat



Decimal Place-Value

-4.25

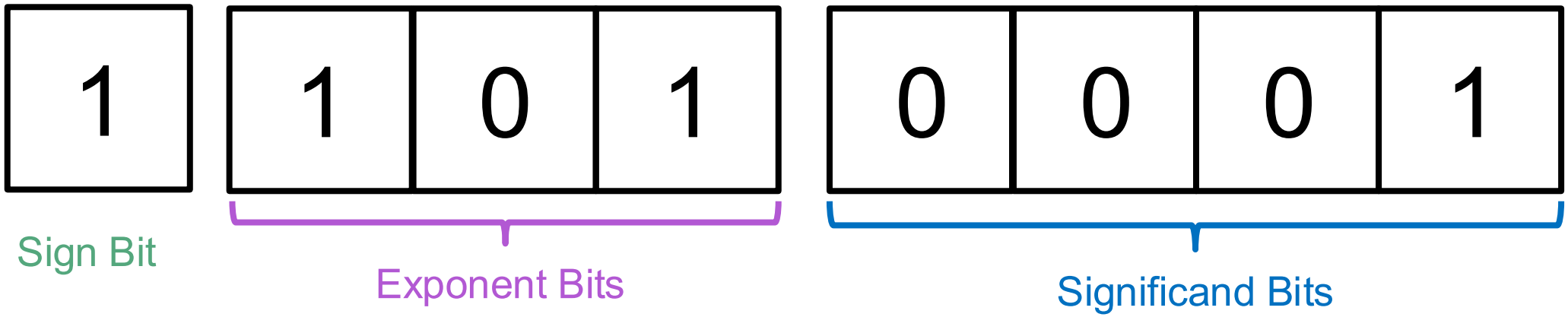
Binary Place-Value

-100.01_2

Normalized Form

$-1.0001_2 \times 2^2$

Minifloat



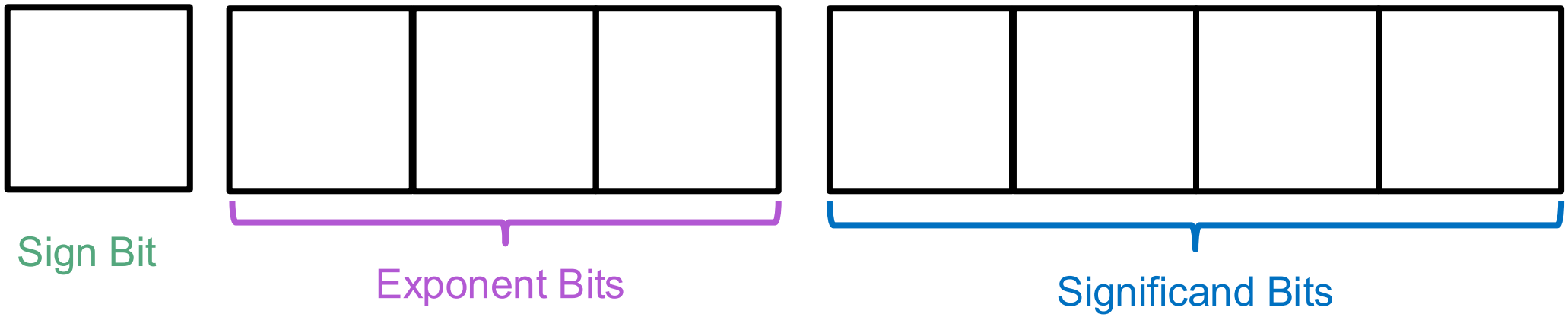
Decimal Place-Value

-32

Binary Place-Value

Normalized Form

Minifloat



Decimal Place-Value

-32

Binary Place-Value

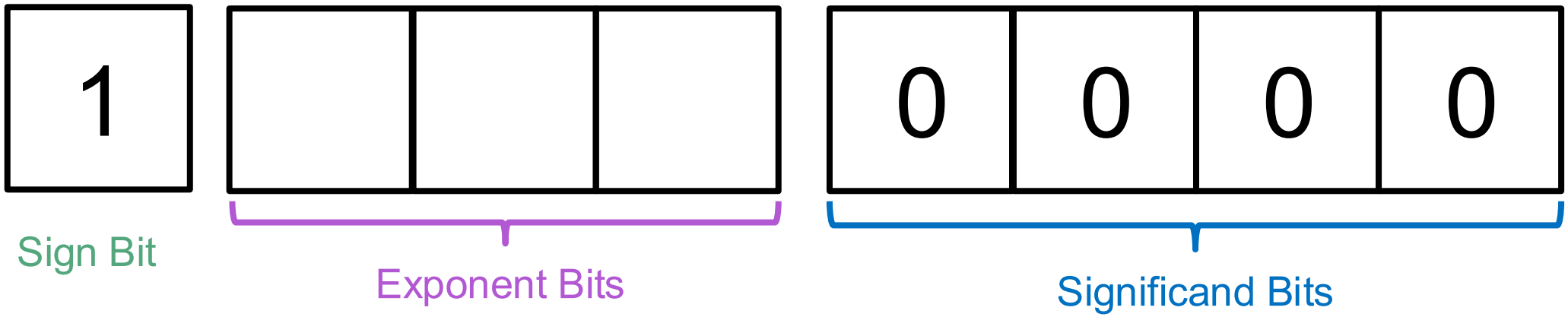
-100000_2

Normalized Form

-1×2^5

5 + 3 = 8 = 0b1000 -> too large for exponent field
Not enough range!

Minifloat



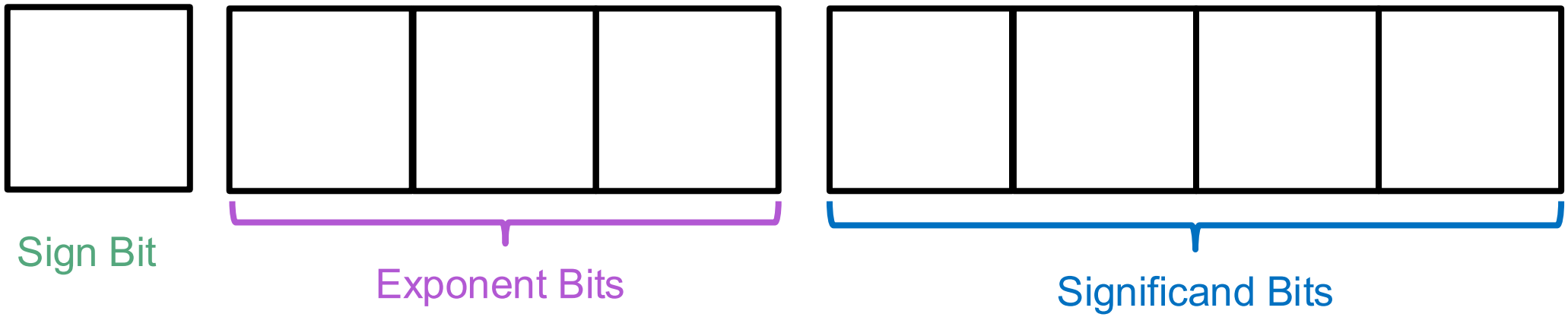
Decimal Place-Value

12.25

Binary Place-Value

Normalized Form

Minifloat



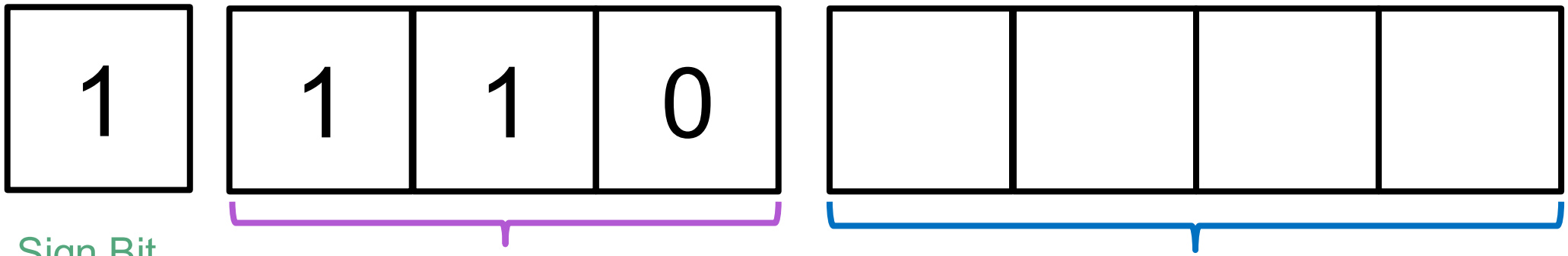
Decimal Place-Value 12.25

Binary Place-Value 1100.01_2

Normalized Form -1.10001×2^3

Need 5 bits for significand
Not enough precision!

Minifloat



Special Cases

Special Cases

- We'll reserve a few bit patterns for special cases

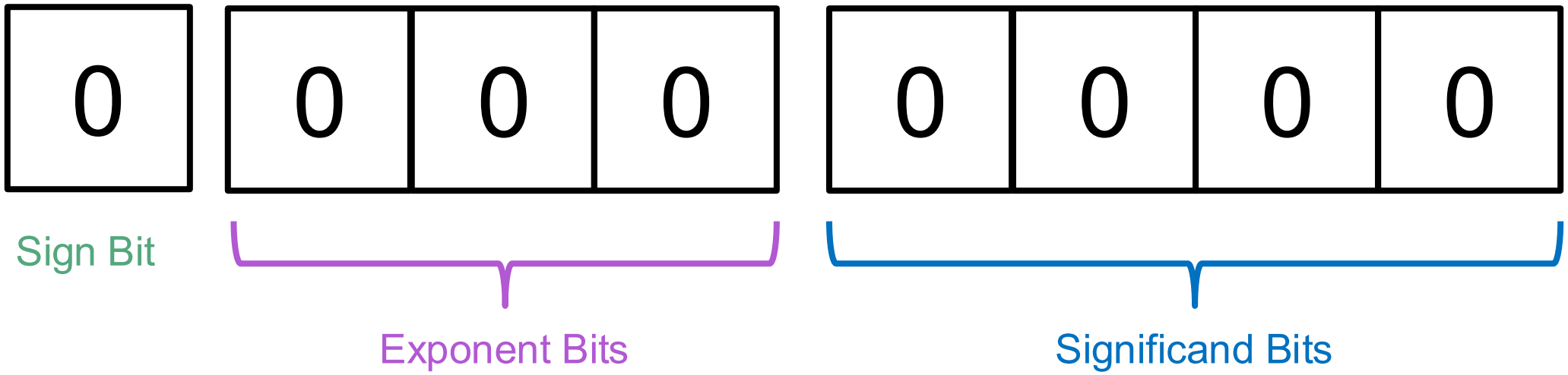
Value	Exponent Bits	Significand Bits
Normalized #	Nonzero	Anything

Special Cases

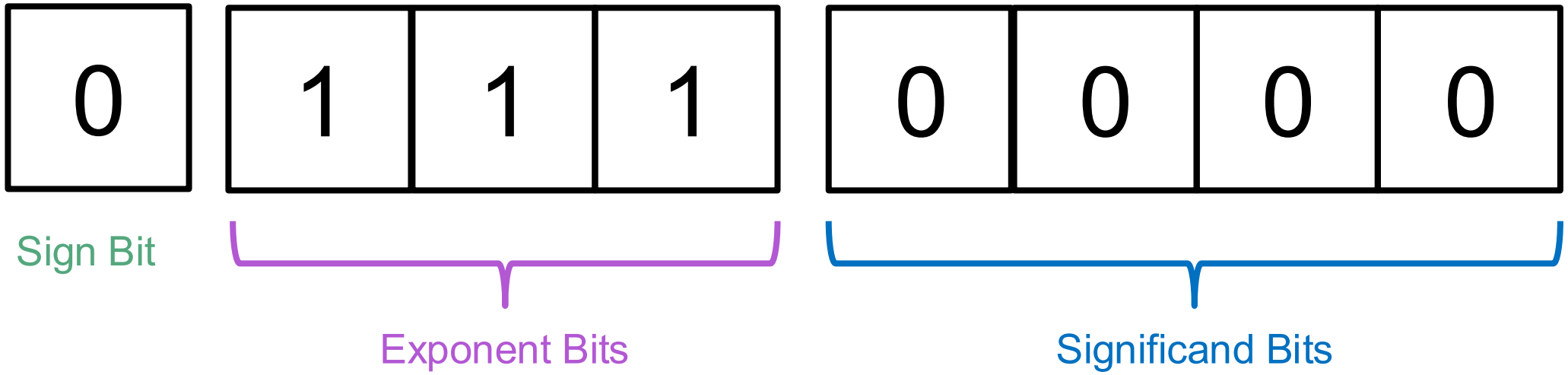
- We'll reserve a few bit patterns for special cases

Value	Exponent Bits	Significand Bits
Normalized #	Nonzero	Anything
Zero	All zeros	All zeros
Infinity	All ones	All zeros

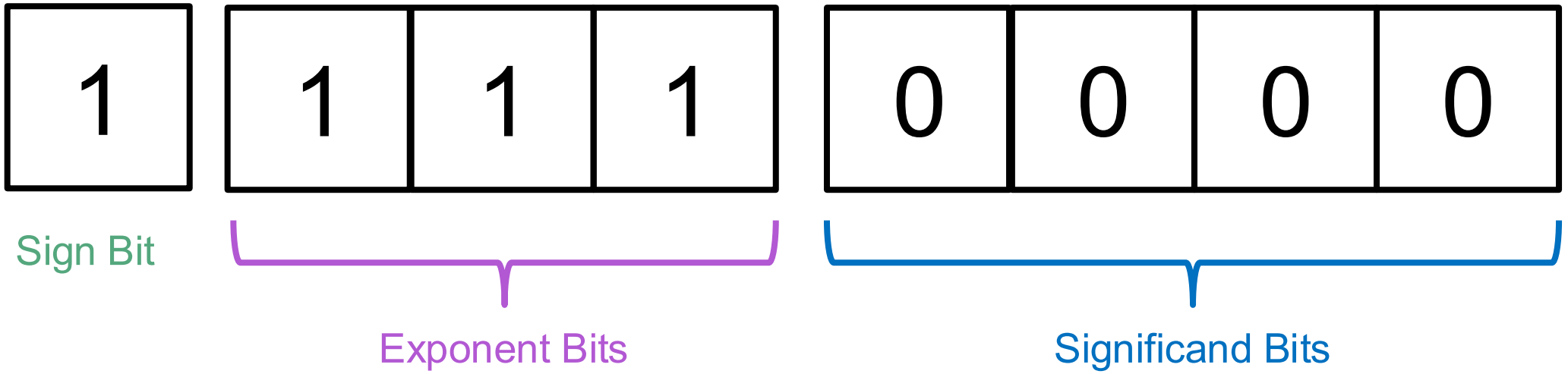
Zero



Positive Infinity



Negative Infinity



Special Cases

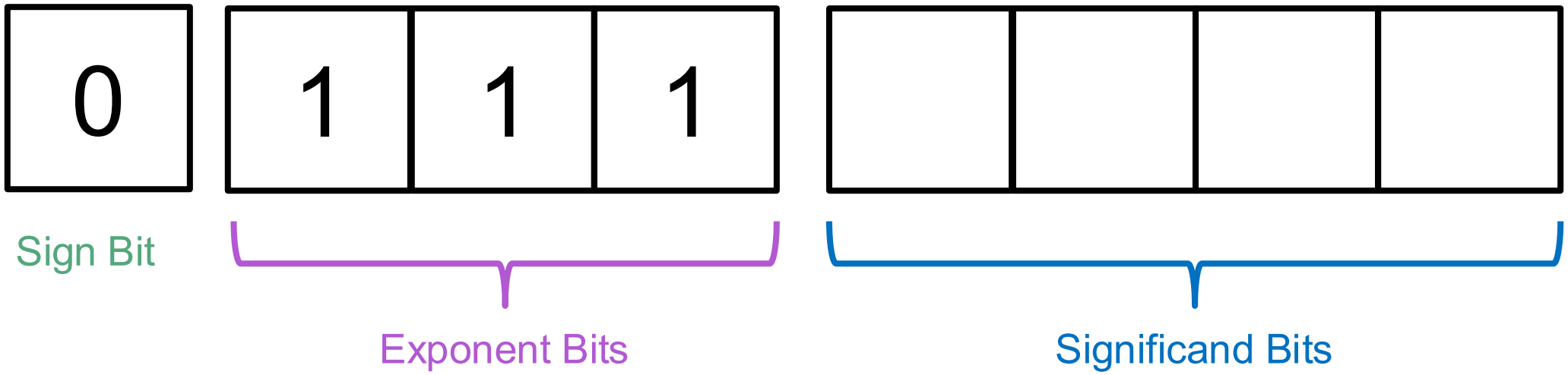
- We've seen Zero and Infinity

Value	Exponent Bits	Significand Bits
Normalized #	Nonzero	Anything
Zero	All zeros	All zeros
Infinity	All ones	All zeros
NaN	All ones	Nonzero

NaN (Not a Number)

- The result of undefined or unrepresentable operations
  - Example operations that would produce NaN result
    - 0/0
    - ∞/∞
    - $\sqrt{-1}$
    - $\log(-5)$
- NaN propagates through calculations
  - Ex: if x = NaN, then x + 5 = NaN
  - Enables graceful handling of errors
  - Helpful for debugging

NaN



Significand is any non-zero value

Why not just one NaN?

- Different NaN encodings let programs distinguish between error sources.

Convert Each Minifloat Value to Decimal, Bias = 3

1. 0 101 0110
2. 1 001 1000
3. 1 111 0000
4. 0 111 0101

Floating Point Rounding

What happens when a number cannot be represented?

- If a floating-point number cannot be represented exactly, the computer generally stores the closest representable floating-point value instead.

- If the number is within the representable range, but not exactly representable
  - It gets rounded to a nearby representable value.
- The number is too large
  - If the magnitude exceeds what the floating-point format can represent, you get **overflow**.
  - The value is encoded as infinity or negative infinity.
- The number is extremely close to zero
  - You can get **underflow** where the value rounds to zero.

Floating Point Rounding

- We cannot represent -32 in minifloat because it is outside of the range of representable values. In this case, the computer would encode negative infinity.

- Minifloat value for -32: 0b1 111 0000 => negative infinity

Floating Point Rounding: 12.25

- **Decimal:** 12.25
- **Binary:** 0b1100.01
- **Normalize:** $1.10001_2 \times 2^3$
- **Minifloat:** $0\text{b}1.\boxed{10001}$
- But we only have 4 fraction bits. So 10001 does not fit exactly.

Floating Point Rounding: 12.25

- **Decimal:** 12.25
- **Binary:** 0b1100.01
- **Normalize:** $1.10001_2 \times 2^3$
- **Minifloat:** $0\text{b}1.\boxed{10001}$
- But we only have 4 fraction bits. So 10001 does not fit exactly.
- The two closest representable values are:
  - $1.1000_2 \times 2^3 = 12.0$ and $1.1001_2 \times 2^3 = 12.5$
- And 12.25 is **exactly halfway** between them.
- Using **round-to-nearest, ties-to-even**, we choose **12.0**.
  - So: 12.25→12.0

Floating Point Rounding: 5.3125

- **Decimal:** 5.3125
- **Binary:** 0b101.0101
- **Normalize:** $1.010101_2 \times 2^2$
- **Minifloat:** $0\text{b}1.\boxed{010101}$
- But we only have **4 fraction bits**. So 010101 does not fit exactly.

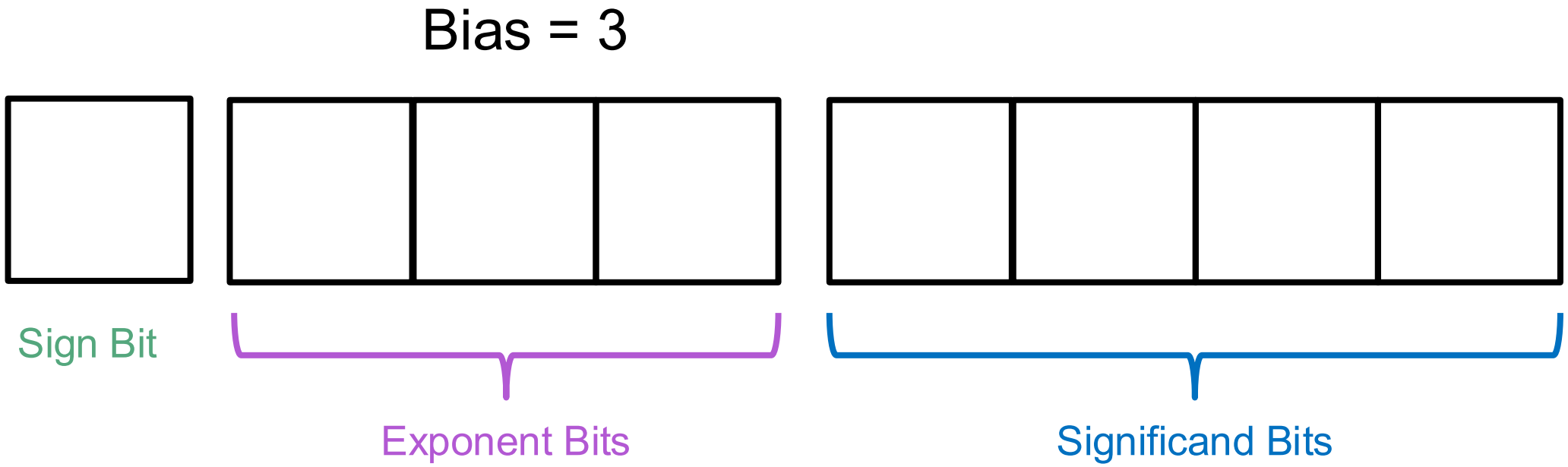
Floating Point Rounding: 5.3125

- **Decimal:** 5.3125
- **Binary:** 0b101.0101
- **Normalize:** $1.010101_2 \times 2^2$
- **Minifloat:** $0\text{b}1.\boxed{010101}$
- But we only have **4 fraction bits**. So 010101 does not fit exactly.
- The two closest representable values are:
  - $1.0101_2 \times 2^2 = 5.25$ and $1.0110_2 \times 2^2 = 5.5$
- And 5.3125 is **closer to 5.25**:
  - $5.3125 - 5.25 = 0.0625$
  - $5.5 - 5.3125 = 0.1875$
- Using **round-to-nearest**, we choose **5.25**.
  - So: 5.3125→5.25

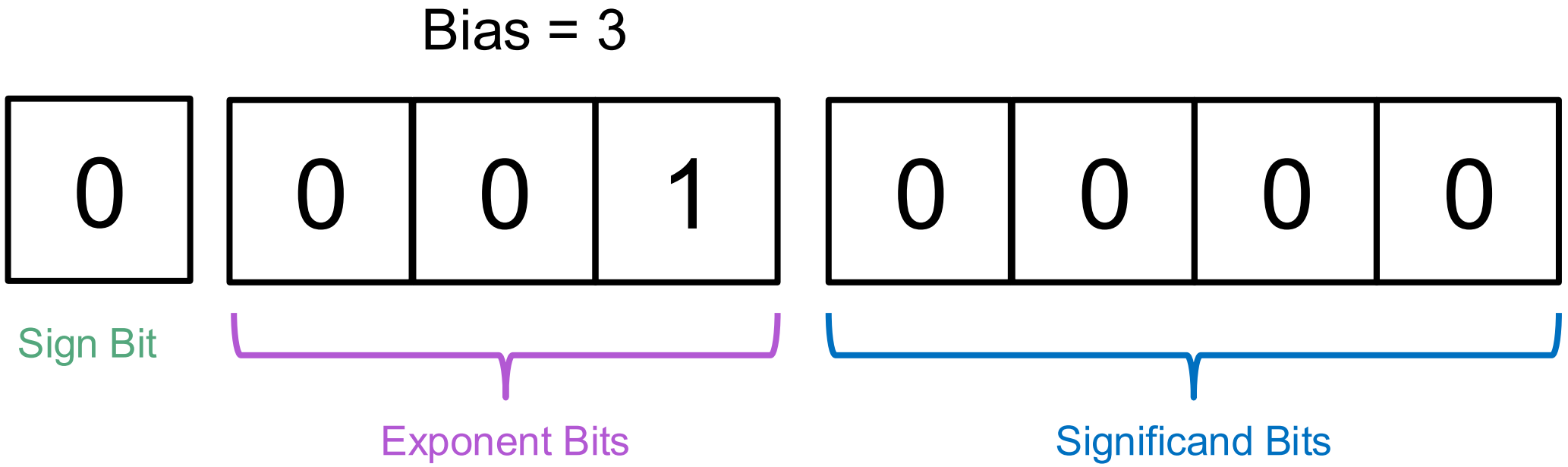
How does 5.125 get represented in minifloat?

Denormalized Numbers

What is the **Smallest** Positive Normalized Value?



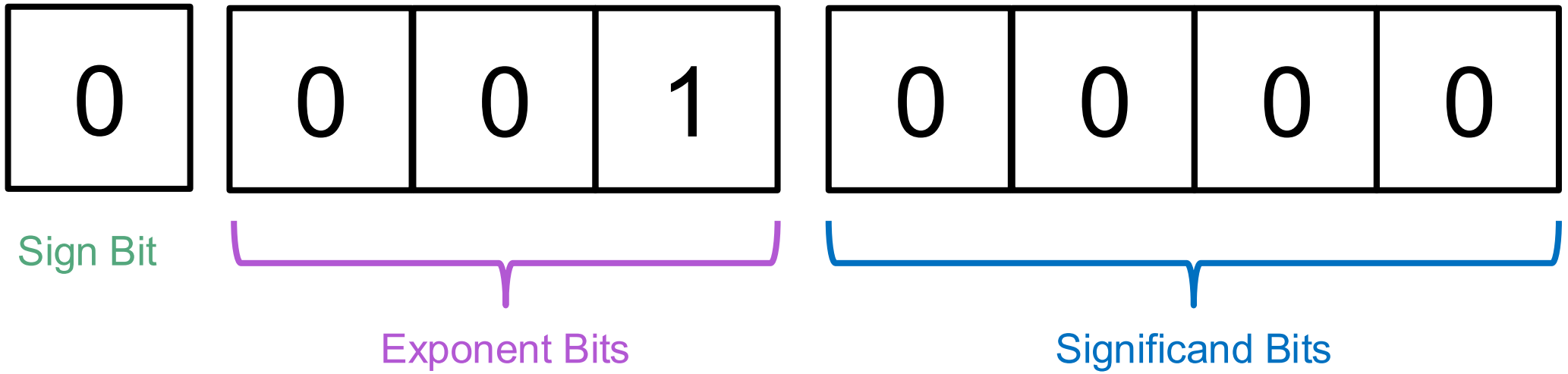
What is the **Smallest** Positive Normalized Value?



$(-1)^{S} \times 1.\langle \text{significand bits} \rangle \times 2^{\langle \text{exponent bits} \rangle - \langle \text{bias} \rangle}$

What is the **Smallest** Positive Normalized Value?

Bias = 3

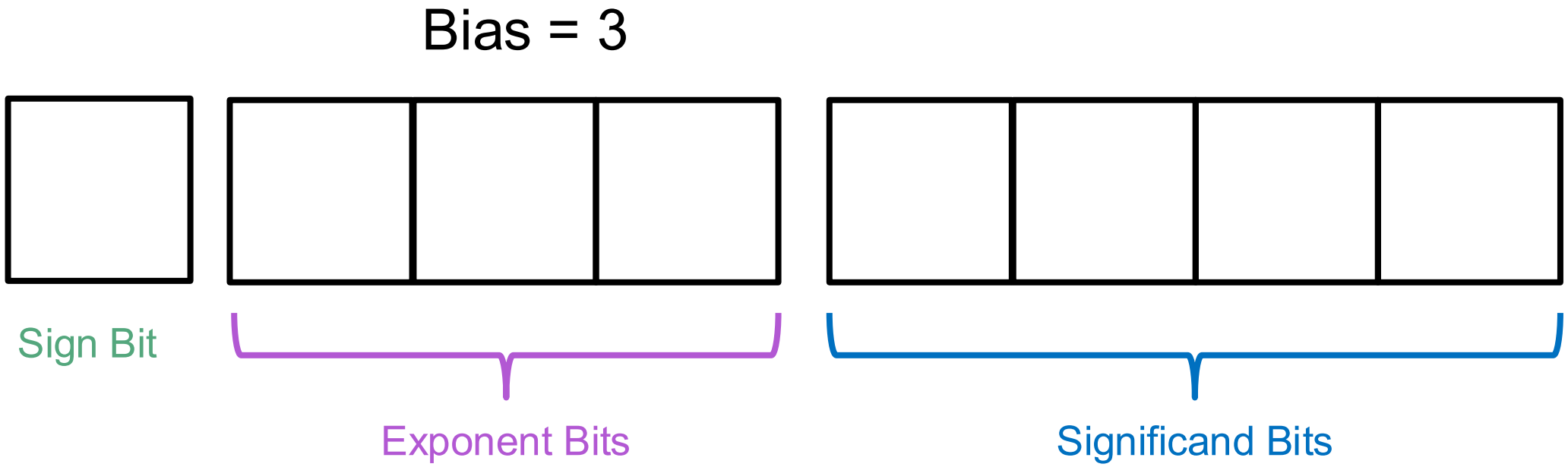


$(-1)^{S} \times 1.\langle \text{significand bits} \rangle \times 2^{\langle \text{exponent bits} \rangle - \langle \text{bias} \rangle}$

$(-1)^{0} \times 1.0000 \times 2^{1-3}$

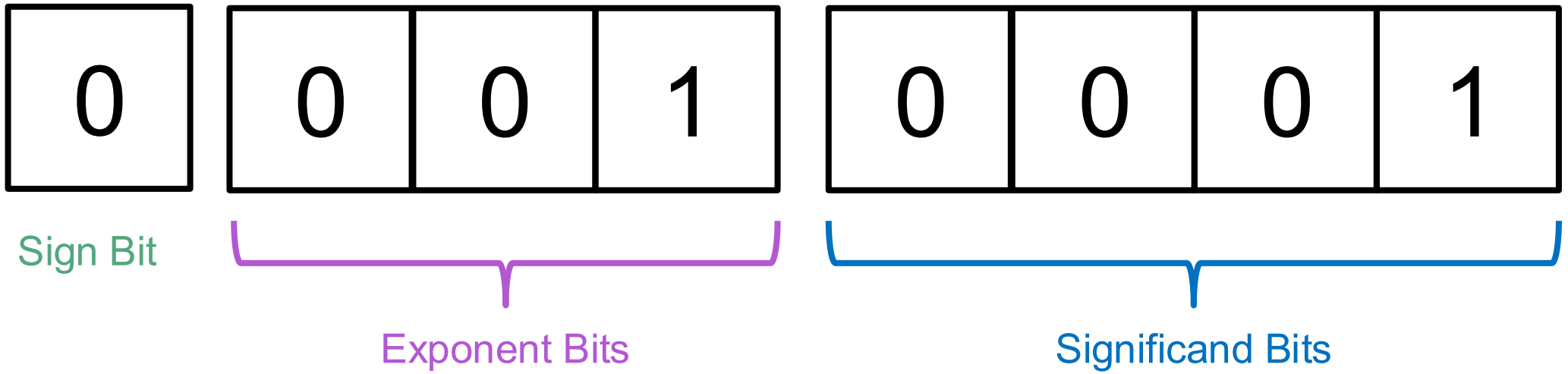
$.01_2 = 0.25$

What is the **Next Smallest** Positive Normalized Value?



What is the **Next Smallest** Positive Normalized Value?

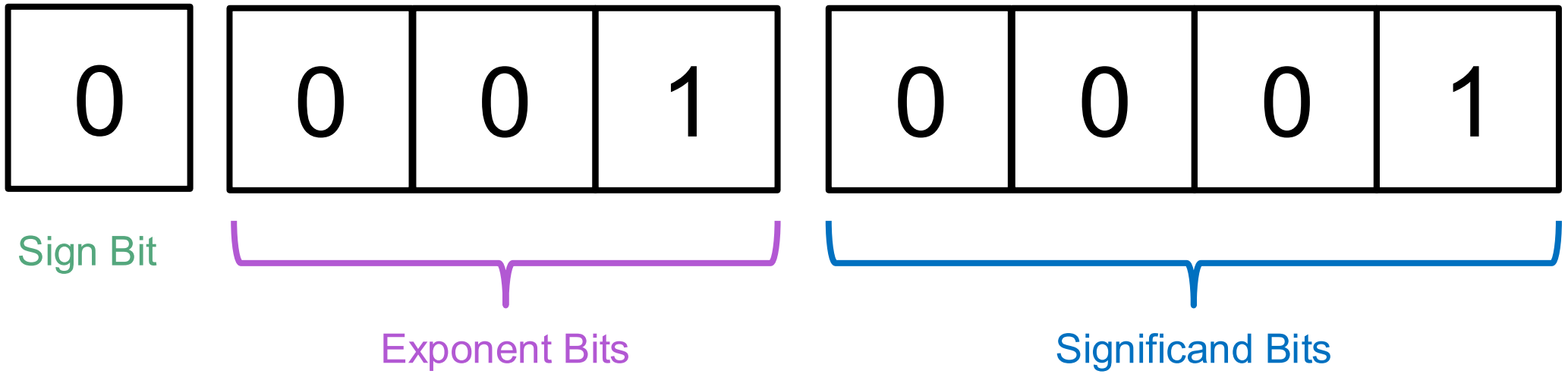
Bias = 3



$(-1)^{S} \times 1.\langle\text{significand bits}\rangle \times 2^{\langle\text{exponent bits}\rangle - \langle\text{bias}\rangle}$

What is the **Next Smallest** Positive Normalized Value?

Bias = 3

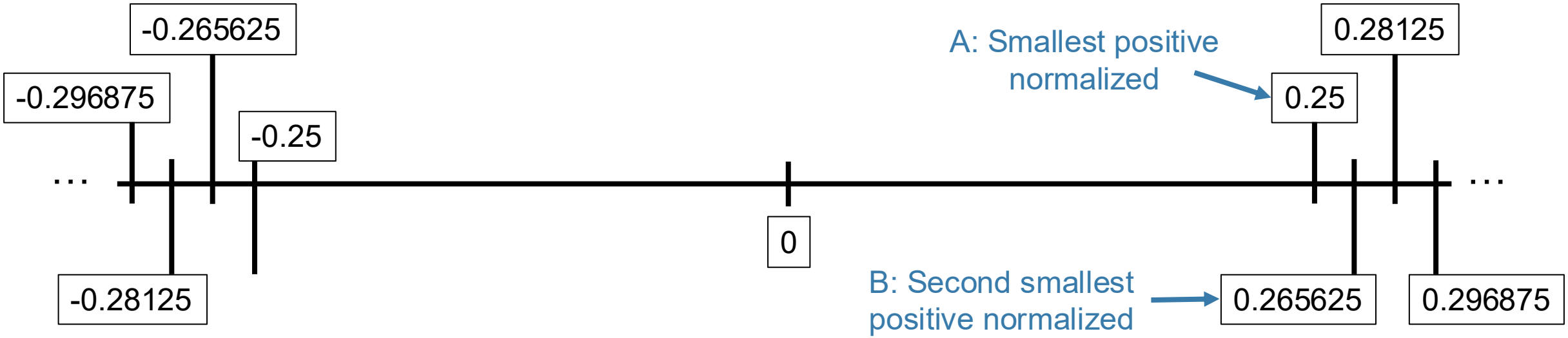


$$(-1)^{S} \times 1.\langle \text{significand bits} \rangle \times 2^{\langle \text{exponent bits} \rangle - \langle \text{bias} \rangle}$$

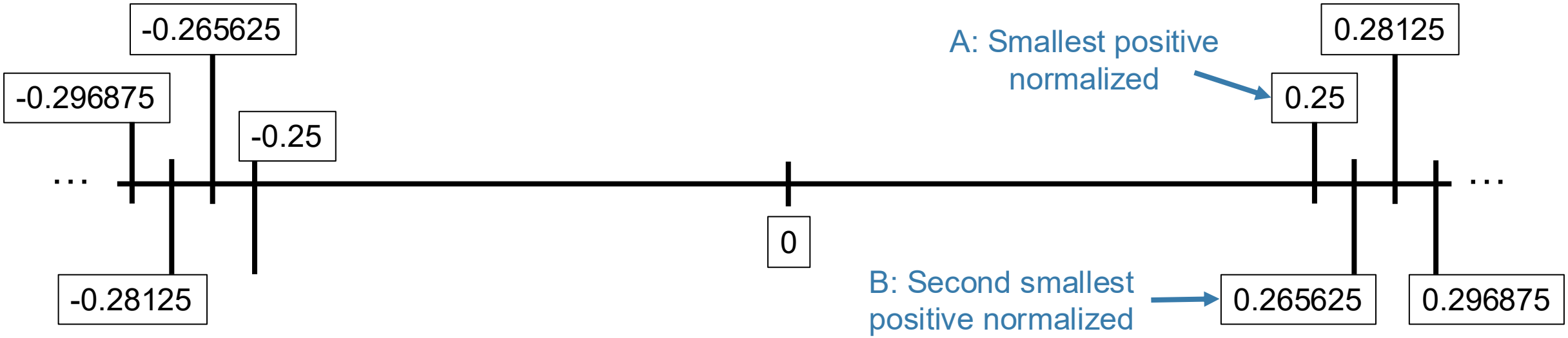
$$(-1)^{0} \times 1.0001 \times 2^{1-3}$$

$$.010001_2 = 2^{-2} + 2^{-6} = 0.265625$$

Number Line of Representable Values



Number Line of Representable Values



Why is the distance between 0 and A so much larger than the distance between A and B?

The implicit leading 1!

Implicit 1

- Remember, implicit 1 means more precision
  - Not storing the leading one gives us room to store an extra bit in the significand
- But this also leaves a large gap between 0 and the smallest non-zero value
  - Any result smaller than the smallest normalized number would **underflow** directly to zero

- **Design trade-off**
  - Pro: Higher precision for the same number of bits
  - Con: A large gap around 0
- Solution: Add Denormalized numbers
  - Drop the hidden 1 to add values around 0

Denormalized Exponent

- The denormalized exponent is always fixed

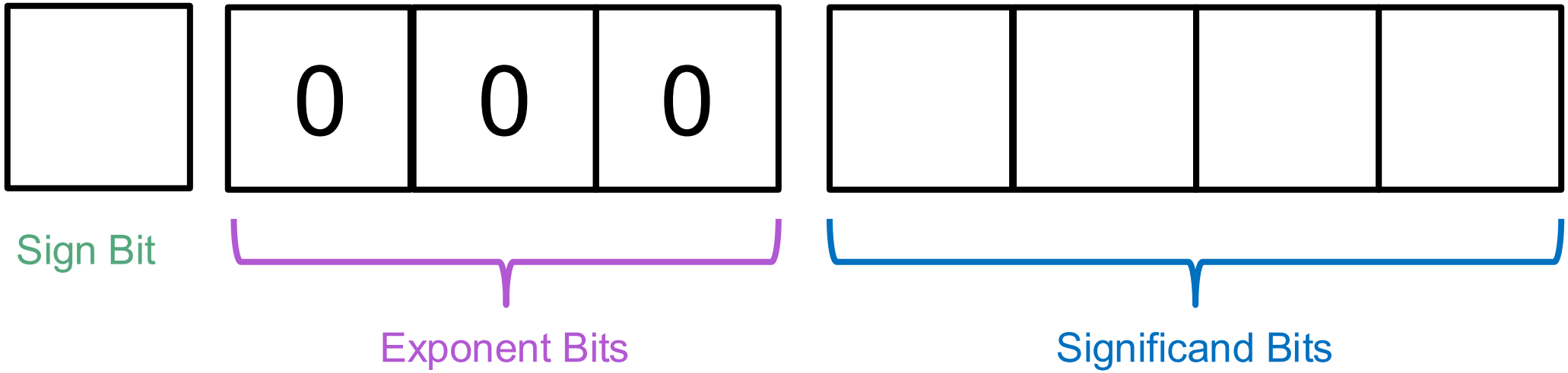
Denorm Exponent = 1 - Bias, where n is the number of exponent bits

Bias = $2^{n-1}-1$

- Minifloat denorm exponent
  - $1 - 2^{n-1}-1$
  - $1 - 2^{2}-1$
  - -2

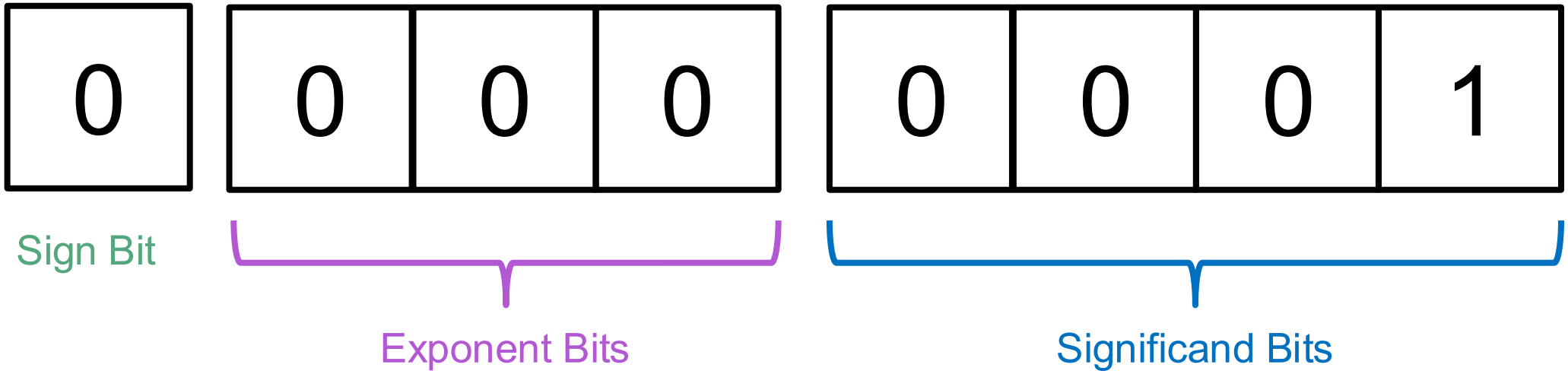
Denormalized Numbers

- To encode a denormalized number, all exponent bits are set to 0

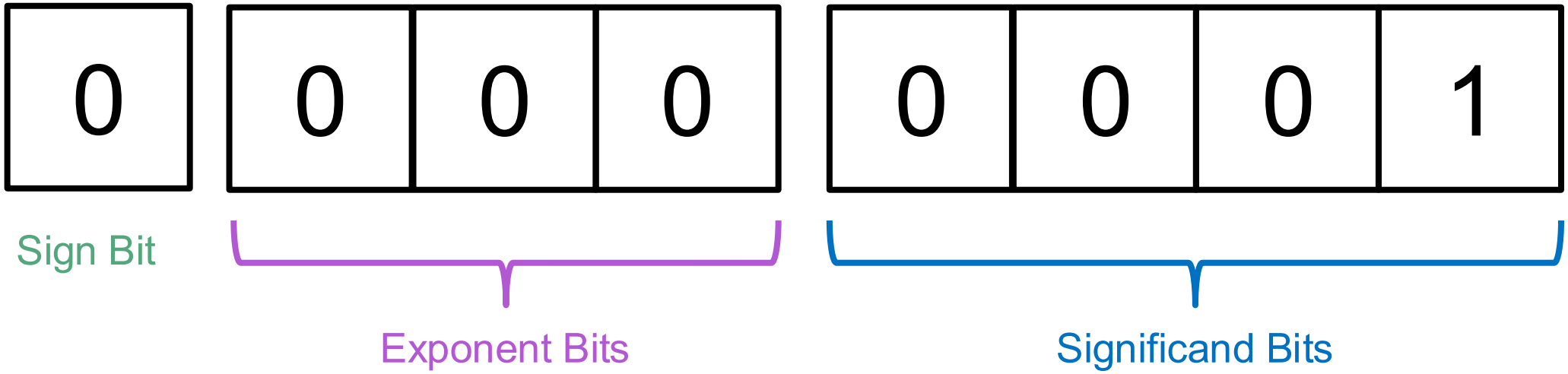


$(-1)^S \times 0.\langle\text{significand bits}\rangle \times 2^{-2}$

Smallest Positive Denormalized Value



Smallest Positive Denormalized Value



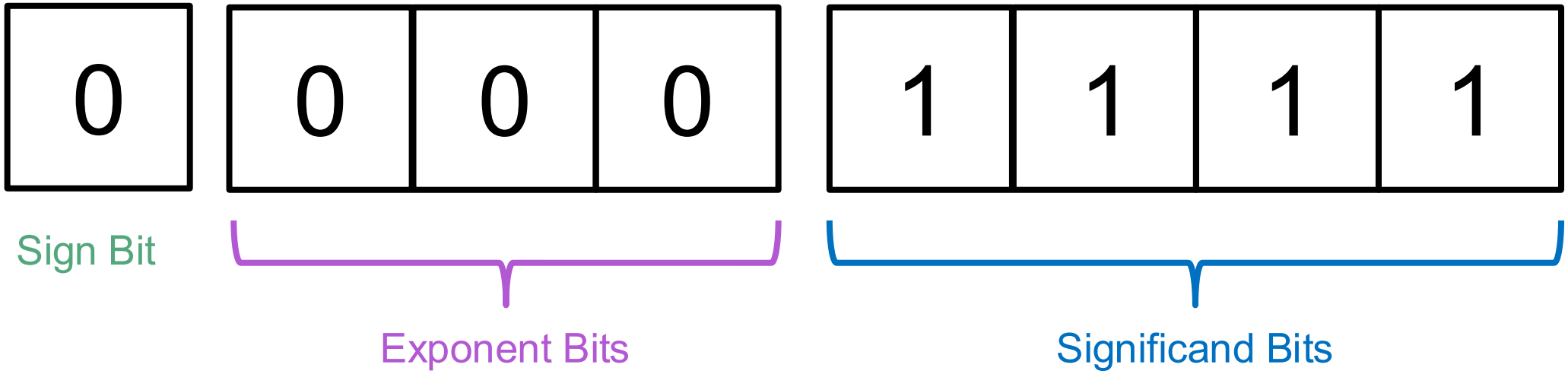
$$(-1)^{S} \times 0.\langle \text{significand bits} \rangle \times 2^{-2}$$

$$(-1)^{0} \times 0.0001 \times 2^{-2}$$

$$(-1)^{0} \times 0.000001$$

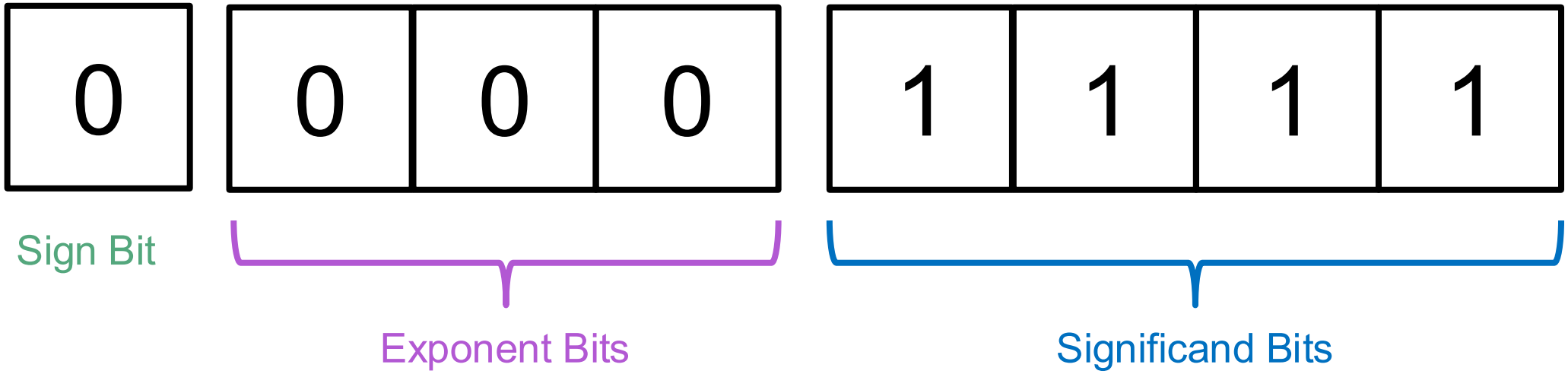
$$2^{-6} = 1/64$$

Example: Denormalized Numbers



$(-1)^{S} \times 0.\langle \text{significand bits} \rangle \times 2^{-2}$

Example: Denormalized Numbers



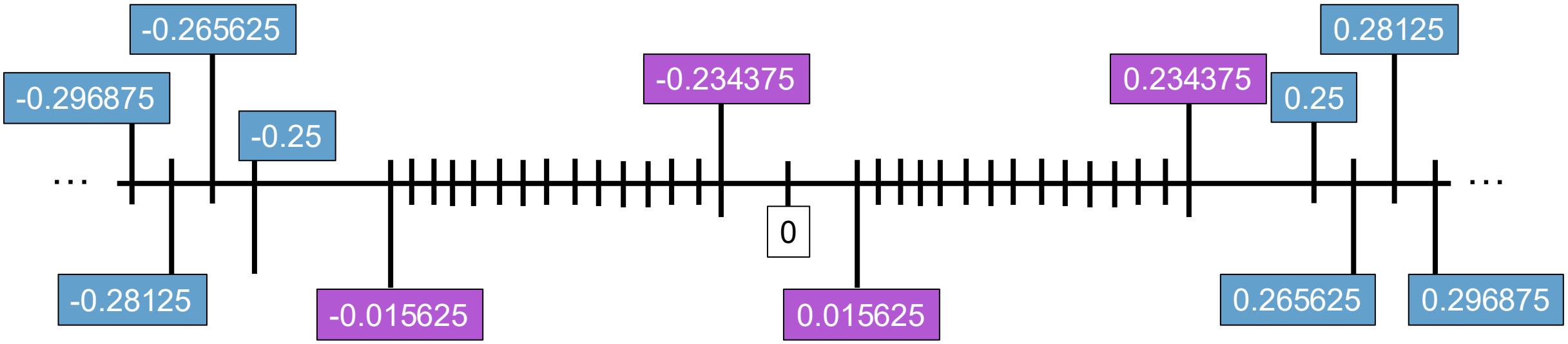
$$(-1)^{S} \times 0.\langle\text{significand bits}\rangle \times 2^{-2}$$

$$(-1)^{0} \times 0.1111 \times 2^{-2}$$

$$(-1)^{0} \times 0.001111$$

15/64

Number Line of Representable Values



= normalized numbers

= denormalized numbers

- **Overflow:** The result of a computation exceeds the largest representable floating-point number
- **Underflow:** A number is too small to be represented and is rounded to zero.

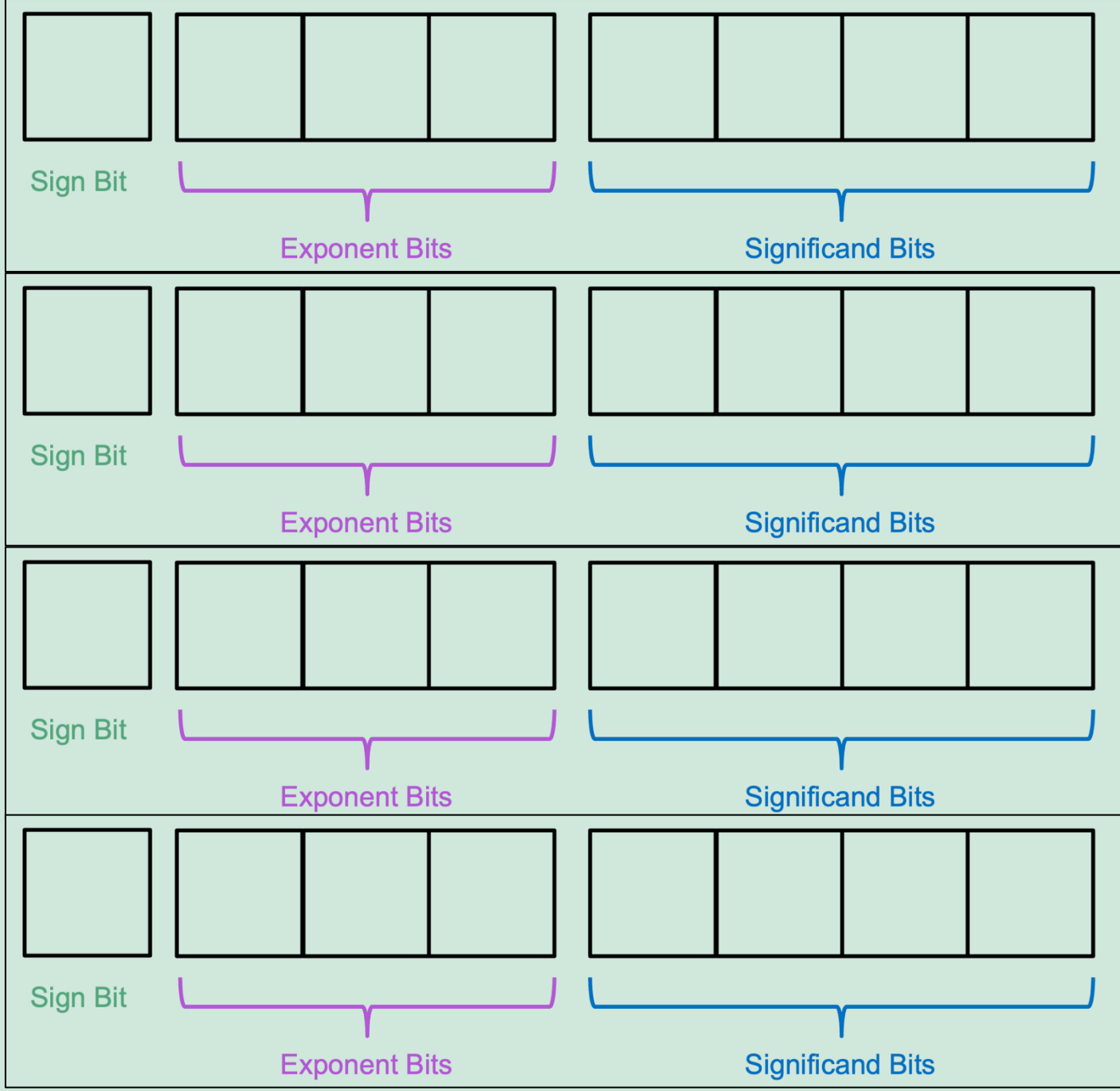
Special Cases

- We've seen denormalized values

Value	Exponent Bits	Significand Bits
Normalized #	Nonzero	Anything
Zero	All zeros	All zeros
Infinity	All ones	All zeros
NaN	All ones	Nonzero
Denormalized #	All zeros	Nonzero

Minifloat

- For each value below, indicate whether it will be represented as a normalized or denormalized value. Then write out its encoding. Hint: first try encoding as a normalized number. If the exponent + bias is <= 0, it is too small to be a normalized number, so try dernorm.
- A) 1/32
- B) -3/8
- C) 9/64
- D) -48/64



Notation

